Task 6

This is the report for task 6 where we implemented load testing for our backend service using Apache JMeter.

Test Setup

We decided to test out our user state endpoints. We chose to test this part of our backend because in TASK2 we wanted to implement a cache for this endpoint to improve performance. So we decided to test this endpoint to see how bad it was and to then see if a cache would be a good way to improve performance.

The endpoint we tested was **/api/test** which will get the user state of the first created user.
The number of **threads** we used was 50 with a **ramp-up period** of 10 seconds.
And an infinite loop count.

We decided to run the test 3 minutes each to see how the system would respond.
We ran both tests simultaneously locally and on Azure.

Locally we ran the service as a normal program without kubernetes.
And on azure we had one node setup with an Autoscaler (HPA) set to the following.

minReplicas: 1
maxReplicas: 5
averageUtilization: 70

Key observations

The local test with no load balancing had a delay on average of 110ms. One can see a clearly linear step up for the first second then it settles for about 110ms. The graph can be viewed in Figure 1.

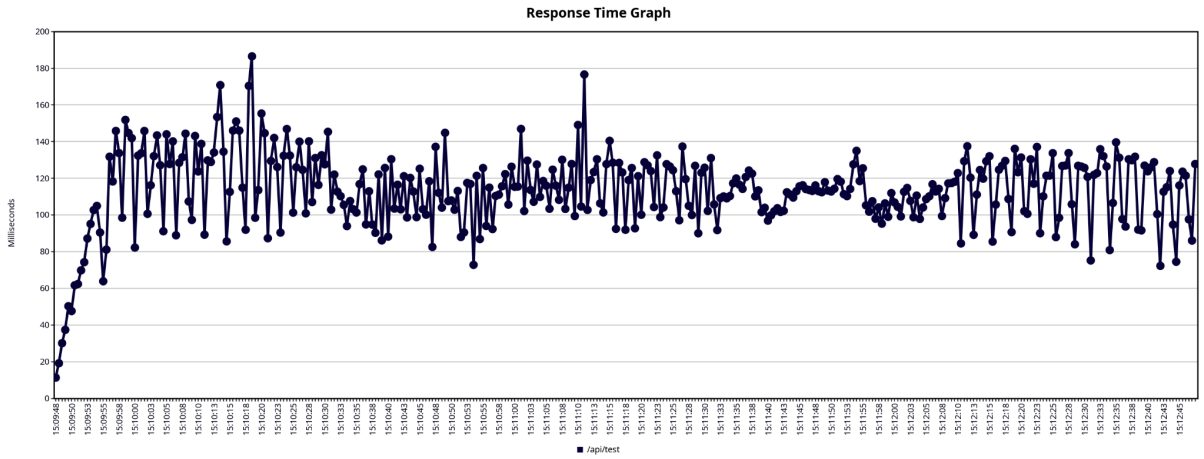

Figure 1

For the Azure test the average delay of the requests was about 55ms. However after about 10 seconds we see some small peaks for about 180ms which is followed 30 seconds later with a higher peak for about 10 seconds where the request delay increases to about 200-600ms. And after it goes back to an average for about 60ms but declining and with an end of about 45ms . The graph can be viewed in Figure 2.

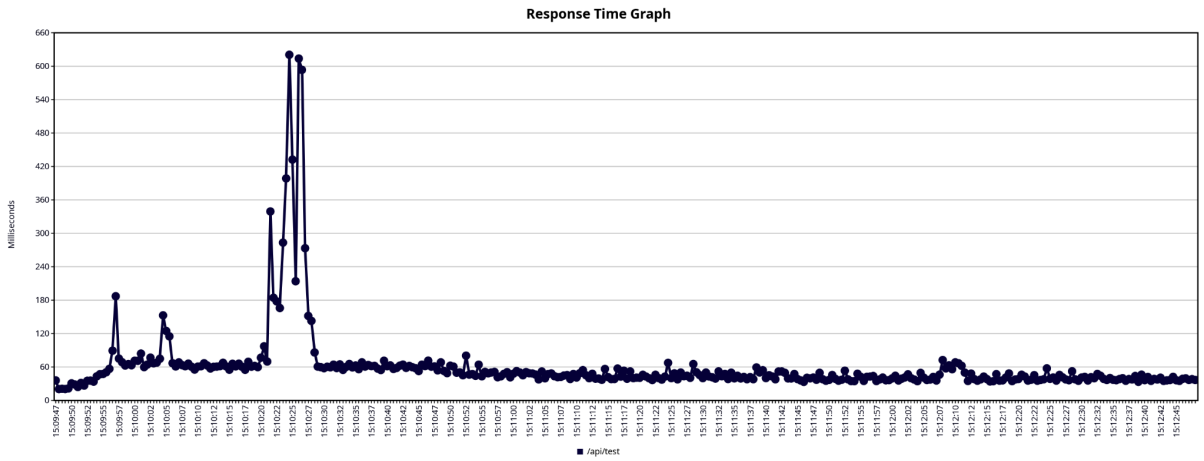

Figure 2

When we observed the kubernetes pods we could see that the HPA was autoscaling and all of our pods were deployed during the test. As viewed in Figure 3 all the pods are deployed and the age can be viewed with the resource usage.

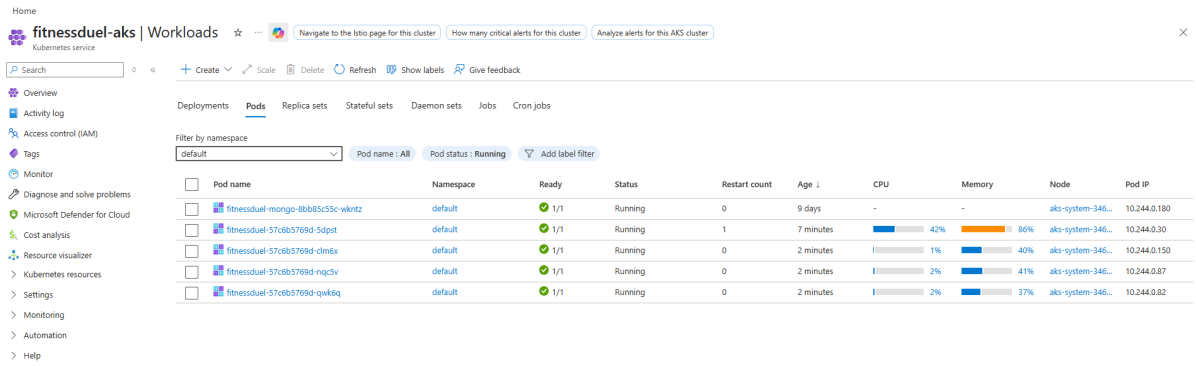

Figure 3

Analysis and reflection

The endpoints, especially the Azure one, indicated great performance at 60ms and 110ms and doesn't need any improvement at this stage. But as we saw, the delay increases when the number of users increases and should be observed if further users were to connect.

The great peak, which can be observed in Figure 2, is very interesting. Our hypothesis is that it is when the autoscaler deploys the other pods which causes some overhead which limits the service making the delay increased.

After the deployment (we are assuming) of more pods we can see in Figure 2 that the delay decreases which means that the system responds faster meaning the system was scaling and performed better.

A key point is also that on Azure we are only using one node, meaning that the hardware limit is still the same even though we are using more pods who can on a software level handle more requests. If we were to introduce more nodes our assumption would be that the delay would decline further.

Lessons learned

The lesson that we learned from this task, is that the load testing can help us confirm if our architectural improvement (caching) was helpful in practice. This test showed us that automating tests are useful for correctness, while load testing is needed for testing performance under load. Since we tested repeated requests to the same endpoint we intern evaluated the usefulness of cache. This test validated our design choice since we discussed the endpoint in our task 2. We also learned that the system can behave differently locally vs in Azure, since the cloud adds factors such as latency and scaling behavior. And another lesson was that monitoring and HPA are important, because they help explain why performance changes under load and whether scaling actually happens.